// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Rekurencja, która zwraca wynik

Funkcje rekurencyjne z return: suma, silnia, potęga, cyfry liczby, tablice, palindromy, akumulator i NWD. Zobaczysz, jak pytania schodzą w dół stosu, a wyniki wracają w górę — kopia po kopii.

C++ · REKURENCJA · CZĘŚĆ 2 Z 6 · PO CZĘŚCI 1

```
↓ suma(3) = 3 + suma(2)
    ↓ suma(2) = 2 + suma(1)
        ↑ suma(1) = 1
    ↑ = 3
↑ = 6
```

PLAN PREZENTACJI

01 Przypomnienie z części 1
02 return w rekurencji
03 Suma od 1 do n
04 Wynik płynie w górę
05 Zaufaj rekurencji
06 Silnia i potęga
07 Cyfry liczby
08 Rekurencja na tablicy
09 Napisy i dwa indeksy
10 Parametr-akumulator
11 NWD i inne kroki
12 Pułapki
13 Sprawdź się
14 Podsumowanie

01 Co już umiemy

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}
```

1. **Warunek stopu** — najmniejszy przypadek, w którym funkcja nie woła siebie.
2. **Mniejszy problem** — wywołanie z argumentem bliższym warunkowi stopu.
3. Kod **przed** wywołaniem działa w drodze w dół, kod **po** — w drodze w górę.
4. Każde wywołanie to osobna **kopia** funkcji na **stosie**, z własnymi zmiennymi.

01 Dziś: funkcja, która coś oblicza

W części 1 funkcje tylko **wypisywały**. Teraz napiszemy funkcje, które **liczą wynik** i oddają go przez **return** — bez ani jednej pętli.

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}

int main() {
    cout << suma(5);
}
```

NA EKRANIE

```
15
```

Dwie linijki, a funkcja liczy 1 + 2 + 3 + 4 + 5. Na kolejnych slajdach zobaczysz, **skąd** się bierze ta liczba — wywołanie po wywołaniu.

02 return — przypomnienie

```
int podwoj(int x) {
    return 2 * x;
}

int main() {
    int a = podwoj(3);
    cout << a + podwoj(10);
}
```

NA EKRANIE

```
26
```

return robi dwie rzeczy naraz:

1. **kończy** funkcję — dalsze linijki już się nie wykonają,
2. **oddaje wynik** tam, skąd funkcję wywołano.

Wywołanie `podwoj(3)` w programie „zamienia się” w liczbę **6**, a `podwoj(10)` — w **20**. Dlatego na ekranie jest 6 + 20 = 26.

02 Wywołanie w środku wyrażenia

```
int podwoj(int x) {
    return 2 * x;
}

int main() {
    cout << podwoj(podwoj(podwoj(1)));
}
```

NA EKRANIE

```
8
```

Gdy wywołanie stoi w środku innego wyrażenia, komputer musi **najpierw** policzyć to wywołanie, a dopiero potem resztę:

1. `podwoj(1)` = **2**
2. `podwoj(2)` = **4**
3. `podwoj(4)` = **8**

W rekurencji będzie dokładnie tak samo: w `return n + suma(n - 1);` funkcja musi **poczekać** na wynik `suma(n - 1)`, zanim doda do niego n.

03 Problem: suma 1 + 2 + … + n

Chcemy policzyć sumę liczb od 1 do n — **bez pętli**. Popatrz na dwie sumy obok siebie:

n	suma(n)	Zauważ
4	1 + 2 + 3 + 4 = **10**	
5	1 + 2 + 3 + 4 + 5 = **15**	to jest **suma(4) + 5**!
6	1 + 2 + 3 + 4 + 5 + 6 = **21**	to jest **suma(5) + 6**

suma(n) = n + suma(n − 1)

> Każdą sumę da się zapisać przy pomocy **mniejszej sumy**. To jest dokładnie to, czego potrzebuje rekurencja.

03 Trzy pytania dla sumy

Pytanie	Odpowiedź
Kiedy kończę?	gdy n == 0: suma zera liczb to **0** — wiem to od ręki
Jak zmniejszyć problem?	wywołuję `suma(n - 1)`
Moja część?	dodać **n** do wyniku mniejszej sumy
Co zwracam?	`n + suma(n - 1)`

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}
```

Te same trzy pytania co w części 1 — dochodzi tylko czwarte: **co zwracam?**

03 Budowa linijki z return

```
return n + suma(n - 1);
       1   2     3
```

1	`return`	oddaj wynik temu, kto mnie wywołał
2	`n`	moja część pracy — liczba, którą dokładam
3	`suma(n - 1)`	wynik mniejszego problemu — na niego trzeba poczekać

Kolejność wykonywania: najpierw liczy się **suma(n − 1)** (cała, ze wszystkimi swoimi wywołaniami), potem dodawanie, a na końcu return.

03 Warunek stopu też coś zwraca

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}
```

W funkcji **void** warunek stopu tylko kończył pracę. W funkcji **int** musi też **oddać wynik** — wynik dla najmniejszego przypadku.

1. suma zera liczb to **0**,
2. iloczyn zera liczb (np. 0!) to **1**,
3. liczba jednocyfrowa ma **1** cyfrę.

Uwaga: Zły wynik w warunku stopu psuje **wszystkie** wyniki — bo każdy wynik jest z niego zbudowany.

04 W dół: suma(4) czeka na suma(3)

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}

int main() {
    cout << suma(4);
}
```

n	4
warunek	`4 == 0` → **NIE**
liczy	`4 + suma(3)` — ale ile to suma(3)? **Czeka.**

STOS WYWOŁAŃ

suma(4) zaczyna, n = 4 ← wierzch

main() czeka na suma(4)

DNO STOSU

04 W dół: każda kopia czeka

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}

int main() {
    cout << suma(4);
}
```

suma(4)	liczy 4 + ?
suma(3)	liczy 3 + ?
suma(2)	liczy 2 + ?
suma(1)	liczy 1 + ?

STOS WYWOŁAŃ

- **suma(0)** zaczyna, n = 0 ← wierzch
- **suma(1)** 1 + ?
- **suma(2)** 2 + ?
- **suma(3)** 3 + ?
- **suma(4)** 4 + ?
- **main()** czeka

DNO STOSU

04 Dno: suma(0) zwraca 0

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}

int main() {
    cout << suma(4);
}
```

warunek	`0 == 0` → **TAK**
zwraca	**0** — od ręki, bez wołania siebie

STOS WYWOŁAŃ

- **suma(0)** return 0 ← wierzch
- **suma(1)** 1 + ?
- **suma(2)** 2 + ?
- **suma(3)** 3 + ?
- **suma(4)** 4 + ?
- **main()** czeka

DNO STOSU

04 W górę: suma(1) = 1 + 0 = 1

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}

int main() {
    cout << suma(4);
}
```

dostaje	suma(0) = **0**
liczy	`1 + 0` = **1**
zwraca	**1** do suma(2)

STOS WYWOŁAŃ

- **suma(1)** 1 + 0 = 1 → return 1 ← wierzch
- **suma(2)** 2 + ?
- **suma(3)** 3 + ?
- **suma(4)** 4 + ?
- **main()** czeka

DNO STOSU

04 W górę: suma(2) = 2 + 1 = 3

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}

int main() {
    cout << suma(4);
}
```

dostaje	suma(1) = **1**
liczy	`2 + 1` = **3**
zwraca	**3** do suma(3)

STOS WYWOŁAŃ

- **suma(2)** 2 + 1 = 3 → return 3 ← wierzch
- **suma(3)** 3 + ?
- **suma(4)** 4 + ?
- **main()** czeka

DNO STOSU

04 W górę: suma(3) = 3 + 3 = 6

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}

int main() {
    cout << suma(4);
}
```

dostaje	suma(2) = **3**
liczy	`3 + 3` = **6**
zwraca	**6** do suma(4)

STOS WYWOŁAŃ

suma(3) 3 + 3 = 6 → return 6 ← wierzch

suma(4) 4 + ?

main() czeka

DNO STOSU

04 W górę: suma(4) = 4 + 6 = 10

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}

int main() {
    cout << suma(4);
}
```

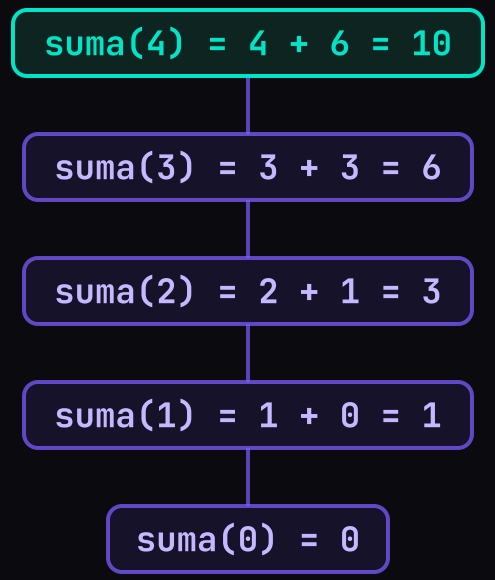
dostaje	suma(3) = **6**
liczy	`4 + 6` = **10**
zwraca	**10** do main, które wypisuje 10

STOS WYWOŁAŃ

main() wypisuje 10 ← wierzch

DNO STOSU

04 Wartości płyną w górę



Pytania idą **w dół** (suma(4) pyta suma(3), ta pyta suma(2)…), a **odpowiedzi wracają w górę** — każda kopia dokłada do odpowiedzi swoje n.

Kopia	Dostaje	Zwraca
`suma(0)`	—	**0**
`suma(1)`	0	1 + 0 = **1**
`suma(2)`	1	2 + 1 = **3**
`suma(3)`	3	3 + 3 = **6**
`suma(4)`	6	4 + 6 = **10**

05 Zaufaj rekurencji

Pisząc suma(n), załóż, że suma(n − 1) już działa i zwraca dobry wynik.

Nie trzeba w głowie schodzić do zera. Wystarczą dwie rzeczy:

1. **warunek stopu** zwraca dobry wynik dla najmniejszego przypadku,
2. **krok** robi dobry wynik dla n z dobrego wyniku dla n − 1.

Wtedy działa **dla każdego n**: suma(0) jest dobra, więc suma(1) jest dobra, więc suma(2) jest dobra, więc suma(3)… — jak domino, w którym każdy klocek przewraca następny.

05 Jak myśleć o funkcji rekurencyjnej?

1 · Co ma zwracać funkcja?

Opisz słowami, np. „suma(n) zwraca 1 + 2 + … + n”. To zdanie jest najważniejsze.

2 · Najmniejszy przypadek

Dla jakiego argumentu znam wynik od ręki? Ten wynik zwróć w warunku stopu.

3 · Załóż, że mniejsze działa

Wolno Ci użyć wyniku dla mniejszego argumentu, tak jakby ktoś już go policzył.

4 · Zbuduj wynik

Jak z wyniku dla mniejszego argumentu zrobić wynik dla n? To jest linijka z return.

06 Silnia

Silnia: **n! = 1 · 2 · 3 · … · n**, a 0! = 1. Na przykład 5! = 120.

n	n!	z poprzedniej
0	1	z definicji
1	1	1 · 0!
2	2	2 · 1!
3	6	3 · 2!
4	24	4 · 3!
5	120	5 · 4!

```
long long silnia(int n) {
    if (n == 0) return 1;
    return n * silnia(n - 1);
}
```

Ten sam schemat co suma, tylko zamiast **+** jest **·**, a warunek stopu zwraca **1**, a nie 0 (pomnożenie przez 0 wyzerowałoby wszystko!).

06 Silnia rośnie bardzo szybko

n	n!	mieści się w
10	3 628 800	int
12	479 001 600	int (ostatnia!)
13	6 227 020 800	**już nie w int**
20	2 432 902 008 176 640 000	long long (ostatnia!)
21	$\approx 5{,}1 \cdot 10^{19}$	**nawet long long za mały**

Uwaga: Dlatego `silnia` zwraca **long long**. Z int dla n = 13 dostalibyśmy bzdurę — liczba „przekręciłaby się" jak licznik.

Zawsze sprawdzaj w treści zadania, jak duży może być wynik, i dobierz typ funkcji.

06 Potęga: $a^n = a \cdot a^{n-1}$

```
long long potega(long long a, int n) {
    if (n == 0) return 1;
    return a * potega(a, n - 1);
}
```

POTEGA(2, 5)

32

↓ potega(2, 5) = 2 · potega(2, 4)
↓ potega(2, 4) = 2 · potega(2, 3)
↓ potega(2, 3) = 2 · potega(2, 2)
↓ potega(2, 2) = 2 · potega(2, 1)
↓ potega(2, 1) = 2 · potega(2, 0)
■ potega(2, 0) = **1**
↑ = 2 · 1 = **2**
↑ = 2 · 2 = **4**
↑ = **8**
↑ = **16**
↑ = **32**

Dwa parametry: **a** przekazujemy dalej bez zmian, maleje tylko **n**. (W części 6 policzymy potęgę dużo szybciej — dzieląc n na pół.)

06 Mnożenie bez mnożenia

a · b to **a + a + … + a** (b razy), czyli **a · b = a + a · (b − 1)**.

```
long long razy(long long a, int b) {
    if (b == 0) return 0;
    return a + razy(a, b - 1);
}
```

Wywołanie	Zwraca
`razy(7, 0)`	0
`razy(7, 1)`	7 + 0 = 7
`razy(7, 2)`	7 + 7 = 14
`razy(7, 3)`	7 + 14 = **21**

Ten sam wzór co silnia i potęga: **wynik(n) = coś ∘ wynik(n − 1)**. Rozpoznawaj go w zadaniach!

07 Ile cyfr ma liczba?

```
int ileCyfr(long long n) {
    if (n < 10) return 1;
    return 1 + ileCyfr(n / 10);
}
```

Liczba ma **o jedną cyfrę więcej** niż ta sama liczba bez ostatniej cyfry (`n / 10`). A liczba jednocyfrowa ma 1 cyfrę.

Wywołanie	Zwraca
`ileCyfr(4821)`	1 + ileCyfr(482) = 1 + 3 = **4**
`ileCyfr(482)`	1 + ileCyfr(48) = 1 + 2 = **3**
`ileCyfr(48)`	1 + ileCyfr(4) = 1 + 1 = **2**
`ileCyfr(4)`	4 < 10 → **1**

Uwaga: Warunek `n < 10` , a nie `n == 0` ! Z `n == 0 → 0` liczba 0 miałaby zero cyfr, a ma jedną.

07 Suma cyfr

```
int sumaCyfr(long long n) {
    if (n == 0) return 0;
    return n % 10 + sumaCyfr(n / 10);
}
```

SUMACYFR(4821)

15

↓ 4821 → 1 + sumaCyfr(482)
  ↓ 482 → 2 + sumaCyfr(48)
    ↓ 48 → 8 + sumaCyfr(4)
      ↓ 4 → 4 + sumaCyfr(0)
        ■ sumaCyfr(0) = **0**
      ↑ 4 + 0 = **4**
    ↑ 8 + 4 = **12**
  ↑ 2 + 12 = **14**
↑ 1 + 14 = **15**

07 Ile razy występuje cyfra d?

```
int ile(long long n, int d) {
    if (n < 10) return n == d;
    return (n % 10 == d) + ile(n / 10, d);
}
```

W C++ porównanie `n % 10 == d` daje **1**, gdy jest prawdziwe, i **0**, gdy fałszywe. Można je dodawać jak liczby!

Wywołanie	ostatnia cyfra = 0?	Zwraca
`ile(1010, 0)`	0 → **1**	1 + 1 = **2**
`ile(101, 0)`	1 → **0**	0 + 1 = 1
`ile(10, 0)`	0 → **1**	1 + 0 = 1
`ile(1, 0)`	jednocyfrowa: 1 == 0?	**0**

07 Funkcja rekurencyjna woła inną rekurencyjną

```
int sumaCyfr(long long n) {
    if (n == 0) return 0;
    return n % 10 + sumaCyfr(n / 10);
}

int pierwiastek(long long n) {
    if (n < 10) return n;
    return pierwiastek(sumaCyfr(n));
}
```

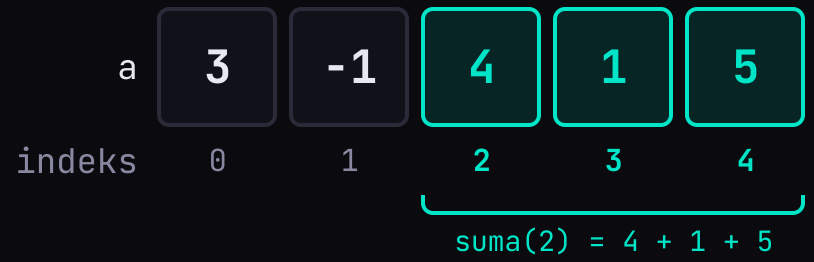
Pierwiastek cyfrowy: sumuj cyfry, aż zostanie jedna cyfra. 9875 → 29 → 11 → **2**.

- ↓ pierwiastek(9875) → sumaCyfr = 29
  - ↓ pierwiastek(29) → sumaCyfr = 11
    - ↓ pierwiastek(11) → sumaCyfr = 2
      - ■ pierwiastek(2): 2 < 10 → **2**

Problem nie musi maleć o 1. Wystarczy, że **na pewno** dojdziemy do warunku stopu — suma cyfr jest mniejsza od liczby (dla n ≥ 10).

08 Indeks jako parametr

Na tablicy też można robić rekurencję. Parametrem jest **indeks**: funkcja zajmuje się kawałkiem tablicy **od indeksu i do końca**.



`suma(i)` = suma elementów od a[i] do końca. Wtedy **suma(i) = a[i] + suma(i + 1)**.

```
int n;
int a[1000];

long long suma(int i) {
    if (i == n) return 0;
    return a[i] + suma(i + 1);
}
```

Z main wołamy `suma(0)` — kawałek od indeksu 0 to cała tablica. Warunek stopu: od indeksu n nie ma już żadnych elementów.

08 Suma tablicy krok po kroku



↓ suma(0) = 3 + suma(1)
↓ suma(1) = −1 + suma(2)
↓ suma(2) = 4 + suma(3)
↓ suma(3) = 1 + suma(4)
↓ suma(4) = 5 + suma(5)
■ suma(5) = **0** (i == n)

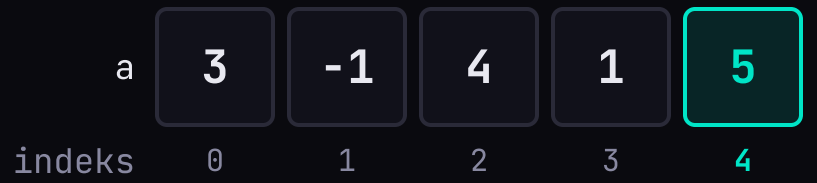
DROGA W GÓRĘ

↑ suma(4) = 5 + 0 = **5**
↑ suma(3) = 1 + 5 = **6**
↑ suma(2) = 4 + 6 = **10**
↑ suma(1) = −1 + 10 = **9**
↑ suma(0) = 3 + 9 = **12**

08 Największy element

```
int maks(int i) {
    if (i == n - 1) return a[i];
    return max(a[i], maks(i + 1));
}
```

Największy z kawałka od i = **większy z dwóch**: a[i] i największy z reszty. Warunek stopu: kawałek z **jednym** elementem — on jest największy.



Wywołanie	Zwraca
`maks(4)`	jeden element → **5**
`maks(3)`	max(1, 5) = **5**
`maks(2)`	max(4, 5) = **5**
`maks(1)`	max(−1, 5) = **5**
`maks(0)`	max(3, 5) = **5**

08 Liczenie elementów spełniających warunek

```
int ileParzystych(int i) {
    if (i == n) return 0;
    if (a[i] % 2 == 0)
        return 1 + ileParzystych(i + 1);
    return ileParzystych(i + 1);
}
```



Jeśli a[i] pasuje — dokładamy **1** do wyniku z reszty tablicy. Jeśli nie — wynik to po prostu wynik z reszty.

ILEPARZYSTYCH(0)

3

08 Funkcja zwracająca prawdę albo fałsz

```
bool posortowana(int i) {
    if (i >= n - 1) return true;
    if (a[i] > a[i + 1]) return false;
    return posortowana(i + 1);
}
```

Funkcja rekurencyjna może zwracać **bool**. Gdy znajdzie zły układ, od razu zwraca **false** — dalszych wywołań już nie ma.



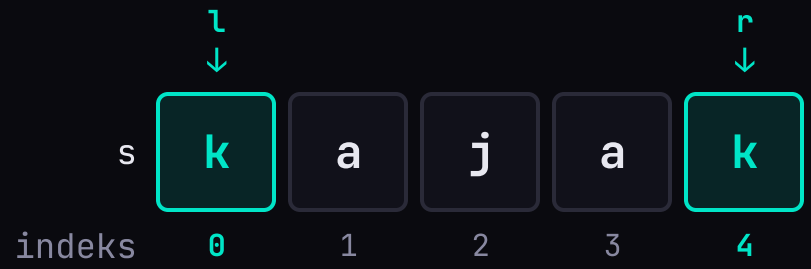
↓ i = 0: 1 ≤ 3, sprawdzam dalej
- i = 1: 3 > 2 → **false**

↑ posortowana(0) zwraca **false**

09 Palindrom — dwa indeksy

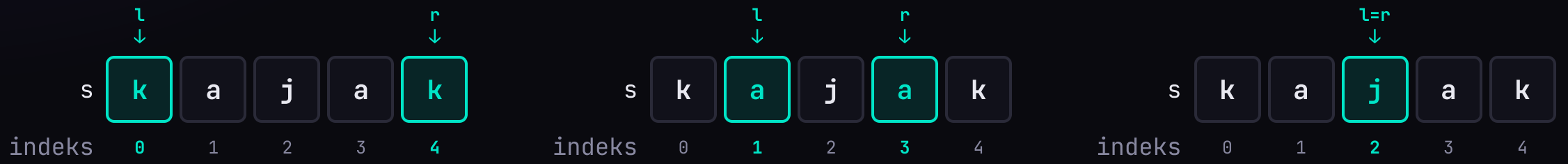
Słowo jest palindromem, gdy czytane od tyłu brzmi tak samo: **kajak**, **radar**. Sprawdzamy **skrajne litery** i idziemy do środka.

```
bool palindrom(const string &s, int l, int r) {
    if (l >= r) return true;
    if (s[l] != s[r]) return false;
    return palindrom(s, l + 1, r - 1);
}
```



Parametry **l** i **r** wyznaczają kawałek słowa, który jeszcze trzeba sprawdzić. Każde wywołanie zwęża go z obu stron.

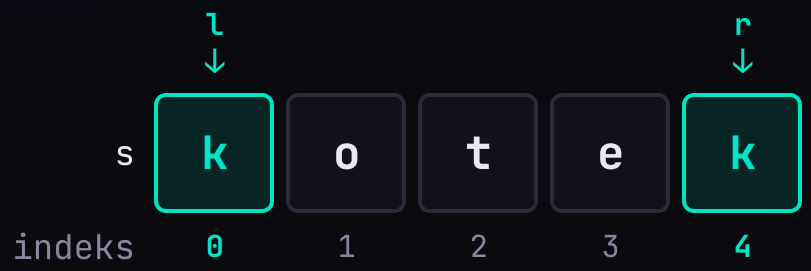
09 Palindrom: kajak

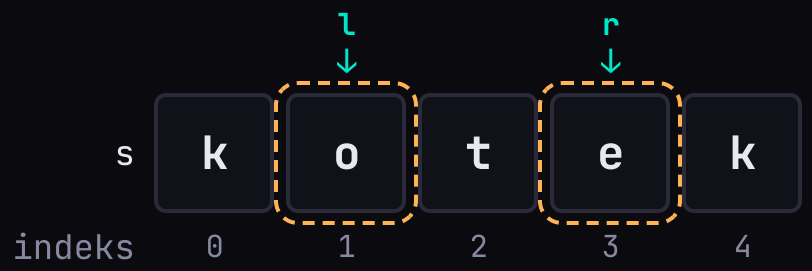


Wywołanie	Porównanie	Wynik
`palindrom(s, 0, 4)`	k == k	sprawdzam środek
`palindrom(s, 1, 3)`	a == a	sprawdzam środek
`palindrom(s, 2, 2)`	l ≥ r — została jedna litera	**TAK** → true

Wynik **true** wraca w górę przez wszystkie kopie bez zmian — każda zwraca to, co dostała.

09 Palindrom: kotek





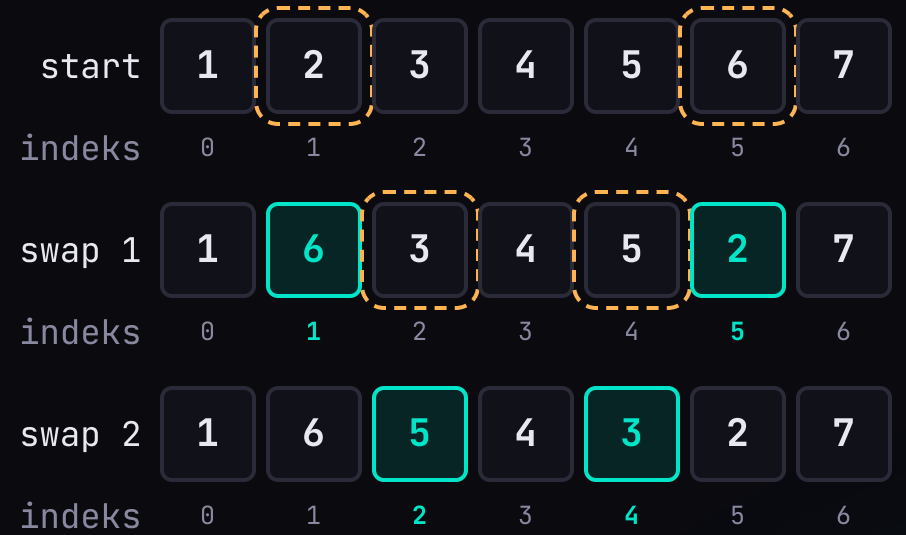
Wywołanie	Porównanie	Wynik
`palindrom(s, 0, 4)`	k == k	sprawdzam środek
`palindrom(s, 1, 3)`	o ≠ e	**NIE** → false

Przy pierwszej różnicy funkcja od razu zwraca **false** — środka „t” nikt już nie ogląda.

09 Odwracanie fragmentu tablicy (swap)

```
void odwroc(int l, int r) {
    if (l >= r) return;
    swap(a[l], a[r]);
    odwroc(l + 1, r - 1);
}
```

Funkcja **void**, ale coś zmienia: tablicę globalną. Po powrocie z **odwroc(1, 5)** fragment jest odwrócony.



odwroc(3, 3): l ≥ r — środek zostaje, koniec.

10 Dwa sposoby liczenia sumy

SPOSÓB 1 — ZWRACAMY WYNIK

```
int suma(int n) {
    if (n == 0) return 0;
    return n + suma(n - 1);
}
```

Wynik **buduje się w drodze w górę**: każda kopia dodaje swoje n do tego, co dostała.

SPOSÓB 2 — WYNIK W PARAMETRZE

```
void suma(int n, int wynik) {
    if (n == 0) {
        cout << wynik;
        return;
    }
    suma(n - 1, wynik + n);
}
```

Wynik **buduje się w drodze w dół** — w parametrze (akumulatorze). Start: `suma(5, 0)`.

10 Akumulator krok po kroku

↓ suma(5, 0) → suma(4, 0 + 5)
  ↓ suma(4, 5) → suma(3, 5 + 4)
    ↓ suma(3, 9) → suma(2, 9 + 3)
      ↓ suma(2, 12) → suma(1, 12 + 2)
        ↓ suma(1, 14) → suma(0, 14 + 1)
          ■ suma(0, 15): wypisuje **15**

Parametr **wynik** to „dotychczas uzbierana suma”. Każde wywołanie przekazuje dalej **większą** sumę, a warunek stopu ma już gotowy wynik.

> Oba sposoby są dobre. W zadaniach z kolejnych części (generowanie, backtracking) parametry-akumulatory będą bardzo częste.

10 Liczba od tyłu z akumulatorem

```
long long odwroc(long long n, long long w) {
    if (n == 0) return w;
    return odwroc(n / 10, w * 10 + n % 10);
}
```

W każdym kroku przenosimy **ostatnią cyfrę** n na **koniec** w. Start: `odwroc(4821, 0)`.

n	w	następne wywołanie
4821	0	odwroc(482, 0·10 + 1)
482	1	odwroc(48, 1·10 + 2)
48	12	odwroc(4, 12·10 + 8)
4	128	odwroc(0, 128·10 + 4)
0	**1284**	stop: zwraca 1284

11 NWD — algorytm Euklidesa

NWD(a, b) — największa liczba dzieląca i a, i b. Starożytny pomysł Euklidesa: **NWD(a, b) = NWD(b, a % b)**, a **NWD(a, 0) = a**.

```
long long nwd(long long a, long long b) {
    if (b == 0) return a;
    return nwd(b, a % b);
}
```

a	b	a % b	wywołanie
48	18	12	nwd(18, 12)
18	12	6	nwd(12, 6)
12	6	0	nwd(6, 0)
6	0	—	stop: **6**

Krok nie jest „n − 1”: liczby maleją dużo szybciej. Nawet dla liczb rzędu 10^{18} wystarczy niecałe 100 wywołań.

11 Krok w dół nie musi być o 1

```
int kroki(long long n) {
    if (n == 1) return 0;
    if (n % 2 == 0) return 1 + kroki(n / 2);
    return 1 + kroki(3 * n + 1);
}
```

Problem Collatza: parzystą liczbę dziel na 2, nieparzystą zamień na 3n + 1. Ile kroków do 1?

↓ 6 → 3
↓ 3 → 10
↓ 10 → 5
↓ 5 → 16
↓ 16 → 8 → 4 → 2 → 1
■ kroki(1) = 0

Kroki(6) = **8**. Funkcja ma **dwa** różne wywołania rekurencyjne, ale za każdym razem wykonuje się tylko **jedno** z nich.

11 Czy to potęga dwójki?

```
bool potegaDwojki(long long n) {
    if (n == 1) return true;
    if (n % 2 == 1) return false;
    return potegaDwojki(n / 2);
}
```

n	co się dzieje
64	64 → 32 → 16 → 8 → 4 → 2 → 1 → **true**
12	12 → 6 → 3: nieparzyste ≠ 1 → **false**
1	od razu **true** ($1 = 2^0$)

Dwa warunki stopu: jeden zwraca **true**, drugi **false**. Funkcja może mieć ich tyle, ile potrzeba.

12 Pułapki (1/2)

01 **Brak return przed wywołaniem.** `n + suma(n - 1);` bez słowa return liczy wynik i… wyrzuca go. Funkcja int bez return zwraca śmieci. Kompilator zwykle ostrzega — czytaj ostrzeżenia!

02 **Wywołanie, którego wynik się marnuje.** `suma(n - 1); return n;` — wywołanie się wykona, ale jego wynik nigdzie nie trafia. Wynik trzeba **użyć** w wyrażeniu.

03 **Zły wynik w warunku stopu.** Silnia z `if (n == 0) return 0;` zwraca zawsze 0 — bo każdy wynik to iloczyn z tym zerem.

12 Pułapki (2/2)

01 **Za mały typ.** Suma, silnia, potęga szybko przekraczają int (ok. $2 \cdot 10^9$). Gdy wynik może być duży — **long long** i w funkcji, i w parametrach.

02 **To samo wywołanie dwa razy.** `if (maks(i + 1) > a[i]) return maks(i + 1);` liczy maks(i + 1) dwa razy, a każde z nich znowu dwa razy… Dla 30 elementów to już ponad miliard wywołań! Zapisz wynik w zmiennej: `int m = maks(i + 1);`.

03 **Warunek stopu tylko dla „ładnych" danych.** Sprawdź najmniejsze przypadki: n = 0, n = 1, tablica z jednym elementem, słowo z jednej litery.

13 Pytanie 1: ile wynosi `f(5)`?

```
int f(int n) {
    if (n == 0) return 0;
    return 2 + f(n - 1);
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 1: 10

↓ f(5) = 2 + f(4)
  ↓ f(4) = 2 + f(3)
    ↓ …
      ■ f(0) = 0
↑ f(5) = 2 + 2 + 2 + 2 + 2 + 0 = **10**

Każda z pięciu kopii dokłada 2. Funkcja liczy po prostu **2 · n**.

13 Pytanie 2: ile wynosi `g(4821)`?

```
int g(int n) {
    if (n < 10) return n;
    return g(n / 10);
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 2: 4 — pierwsza cyfra

↓ g(4821) = g(482)
  ↓ g(482) = g(48)
    ↓ g(48) = g(4)
      ■ g(4) = **4**

Funkcja odcina ostatnie cyfry, aż zostanie jedna — **pierwsza** cyfra liczby. Wynik przechodzi w górę bez zmian.

13 Pytanie 3: ile wynosi `h(84, 36)`?

```
long long h(long long a, long long b) {
    if (b == 0) return a;
    return h(b, a % b);
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 3: 12

a	b	a % b
84	36	12
36	12	0
12	0	stop → **12**

To algorytm Euklidesa: NWD(84, 36) = **12**. Rzeczywiście 84 = 7 · 12 i 36 = 3 · 12.

13 Pytanie 4: znajdź błąd

```
int suma(int n) {
    if (n == 0) return 0;
    suma(n - 1) + n;
}
```

NA EKRANIE

?

Funkcja miała liczyć 1 + 2 + … + n.

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 4: brakuje return

POPRAWNIE

```
int suma(int n) {
    if (n == 0) return 0;
    return suma(n - 1) + n;
}
```

Uwaga: Bez **return** wynik dodawania nigdzie nie trafia, a funkcja int kończy się, nie oddając wyniku. To **błąd** — program może wypisać cokolwiek.

Kompilator ostrzega: *control reaches end of non-void function*. Warto kompilować z `-Wall`, żeby takie ostrzeżenia widzieć.

13 Pytanie 5: ile wynosi `f(5)`?

```
int f(int n) {
    if (n <= 1) return 1;
    return f(n - 1) + f(n - 2);
}
```

NA EKRANIE

?

Uwaga: dwa wywołania w jednej linijce.

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 5: 8

n	f(n)
0	1
1	1
2	f(1) + f(0) = 2
3	f(2) + f(1) = 3
4	f(3) + f(2) = 5
5	f(4) + f(3) = **8**

Najłatwiej liczyć **od dołu**: znając f(0) i f(1), po kolei dostajemy f(2), f(3)… To ciąg prawie jak Fibonacci — funkcjom wołającym siebie **dwa razy** poświęcona jest część 3.

14 Ściąga: rekurencja z wynikiem

Wzór	Przykład	Warunek stopu
`return n + f(n - 1);`	suma 1 + … + n	`f(0) = 0`
`return n * f(n - 1);`	silnia	`f(0) = 1`
`return a * f(a, n - 1);`	potęga a^n	`f(a, 0) = 1`
`return n % 10 + f(n / 10);`	suma cyfr	`f(0) = 0`
`return a[i] + f(i + 1);`	suma tablicy	`f(n) = 0`
`return max(a[i], f(i + 1));`	maksimum tablicy	`f(n - 1) = a[n - 1]`
`return f(s, l + 1, r - 1);`	palindrom	`l >= r → true`
`return f(n / 10, w * 10 + n % 10);`	akumulator	`f(0, w) = w`
`return f(b, a % b);`	NWD	`f(a, 0) = a`

14 Zadania na koniec — Twoja kolej

Zadania są na **t-olimpians.com** → **Kompendium** → **Funkcje rekurencyjne** → **Krok 2**. Każde ma sprawdzarkę i wskazówkę:

1. Suma od 1 do n
2. Silnia
3. Potęga
4. Ile cyfr?
5. Suma cyfr
6. Ile razy występuje cyfra?
7. Pierwiastek cyfrowy
8. Liczba od tyłu
9. Suma tablicy
10. Największy element
11. Ile liczb parzystych?
12. Czy tablica jest posortowana?
13. Palindrom
14. Odwróć fragment tablicy
15. Największy wspólny dzielnik
16. Mnożenie bez mnożenia
17. Problem Collatza
18. Czy to potęga dwójki?

// KONIEC CZĘŚCI 2

Umiesz już liczyć rekurencją!

Warunek stopu zwraca wynik dla najmniejszego przypadku, a krok buduje wynik z wyniku mniejszego problemu. Zrób zadania z kroku 2, a potem część 3: funkcje, które wołają siebie wiele razy.

